CS 327: Compilers
Assignment A2 Report: Lexical Analyzer for **CYLite**

Swayam Borate (23110066) Shardul Junagade (23110297) Tejas Lohia (23110335)

20th February 2026

1 Objective

The goal of Assignment A2 is to build a lexical analyzer using `lex` for our language **CYLite**. This report summarizes the lexer changes we made to support the keywords and comment rules from our language specification, and how we tested the scanner.

What is a lexical analyzer? A lexical analyzer (or *lexer*) is the first stage of a compiler. It reads the source program as a stream of characters and groups them into meaningful units called *tokens* (for example: identifiers, keywords, numbers, string/char literals, operators, and punctuation). The parser does not work with raw characters; it works with this token stream.

In practice, a lexer also handles the "boring but important" parts: skipping whitespace and comments, and reporting lexical errors early (like an unexpected character or an unterminated comment).

2 What we built

We used a `lex` specification to recognize tokens for a C-like core (identifiers, operators, punctuation, numeric/string/char literals) and then extended it to handle the **CYLite** additions.

For convenient testing, we also used a small driver program (`dump_tokens.c`) that repeatedly calls `yylex()` and prints `(token_id, lexeme)` pairs. This helped us validate scanning without depending on full parsing.

3 Summary of changes

Starting from the provided C-like scanner skeleton, we updated the lexer and token definitions to match the **CYLite** spec (see `CYLite_lang_spec.tex`). The main changes were:

- Added **CYLite** keywords: `elif`, `pass`, `try`, `except`, `print`, `range`, `in`, `foreach`.
- Added `#` as an additional single-line comment starter.
- Made invalid characters a hard lexer error (terminate with a clear message).
- Small updates in the `yacc`/Bison file so the new keyword tokens exist (keeps the build consistent and helps for A3).

Diff highlights (rules we actually touched)

Below is a compact list of the lexer/parser-side edits that show up in the diff.

- Added scanning rule for `#` comments.
- Added scanning rules for each new **CYLite** keyword.

- Replaced the generic "discard bad characters" rule with an explicit error + exit.
- Declared the new keyword tokens in the parser and added C prototypes for `yylex`/`yyerror`.
- Adjusted a few grammar/actions (dangling-else handling, pointer counting action position, and removing `ATOMIC` from type qualifiers).

4 Lexer changes (`lex`)

4.1 `#` single-line comments

In **CYLite**, lines starting with `#` are treated as comments (no preprocessor support). We added a rule to consume the full line, similar to `//`:

```
"//".*      { /* consume //-comment */ }
"#".*       { /* consume #-comment */ }
```

4.2 New keywords

We added explicit rules for the new reserved words so they return dedicated token types. This avoids them being scanned as generic identifiers.

```
"elif"      { return ELIF; }
"pass"      { return PASS; }
"try"       { return TRY; }
"except"    { return EXCEPT; }
"print"     { return PRINT; }
"range"     { return RANGE; }
"in"        { return IN; }
"foreach"   { return FOREACH; }
```

4.3 Invalid character handling

Previously, unknown characters were silently discarded, which can hide bugs in inputs. For A2 we changed this to a hard lexer error with an informative message:

```
. {
                sprintf(buff,
                        "***lexing terminated*** [lexer error]: invalid character '%s
    '",
                        yytext);
                yyerror(buff);
                exit(1);
}
```

4.4 Unclosed multi-line comments

The lexer already handled unclosed `/* ... */` comments by reporting an ill-formed comment. We ensured this path terminates execution so that scanning does not continue past a lexer-fatal error.

The error path is triggered only when the end-of-file is reached before seeing the closing `*/`.

5 Token definitions in `yacc` (build consistency)

Even though A2 is focused on scanning, our build produces `y.tab.h` from Bison/Yacc. To keep everything consistent, we declared the new keyword tokens in the parser file:

```
%token ELIF PASS TRY EXCEPT PRINT RANGE IN FOREACH
```

We also added prototypes for `yylex()` and `yyerror()` at the top of the parser file to avoid implicit-declaration issues with the generated C code.

Other parser-side edits seen in the diff

These were not strictly required for tokenization, but they came up while keeping the codebase consistent with **CYLite** and while preparing for A3.

- **Dangling-else:** the statement grammar was refactored into matched/unmatched forms to reduce ambiguity around `else`.
- **Type qualifier cleanup:** `ATOMIC` was removed from the `type_qualifier` production (since it is not part of **CYLite**).
- **Pointer counting action:** the `pointer_decls++` action was moved to trigger after recognizing a full pointer declarator.

Note. Running Bison prints one shift/reduce conflict warning in the current parser skeleton. This does not affect token dumping, and we plan to clean up any remaining conflicts during A3 when we fully flesh out the grammar.

6 How to build and run

The build uses `lex` and `bison`.

- Build: run `make` (inside the `cylite/` folder).
- Token dump (recommended for A2 testing):

```
cd cylite
make
gcc -O2 dump_tokens.c lex.yy.c -o dump_tokens
./dump_tokens tests/keywords.cyl
```

7 Testcases

All lexer-focused tests are placed under `tests/`. We ran each testcase using:

```
./dump_tokens tests/<name>.cyl
```

Interpreting the output. Each output line is `token_id<TAB>lexeme`. For single-character tokens (punctuation/operators), the scanner returns the character itself, so the token id is the ASCII code (e.g., `59` for `;`). For named tokens (keywords/literals), the numeric ids come from the generated `y.tab.h` for this build.

1. `keywords.cyl`

   Goal Ensure all reserved words (including **CYLite** additions) are recognized as keyword tokens rather than identifiers.

   Observed output

```
323     if
258     elif
324     else
328     for
326     while
```

```
327     do
260     try
261     except
265     foreach
264     in
263     range
259     pass
262     print
308     int
312     float
313     double
306     char
317     struct
271     sizeof
332     return
331     break
330     continue
```

Why this is expected

Each word is returned as a keyword token id, confirming that the explicit keyword rules in `yapl.l` take precedence over the generic identifier rule.

2. `identifiers.cyl`

Goal

Verify identifier forms (underscores, digits after the first character) and ensure near-keywords are not treated as keywords.

Observed output

```
266     x
266     _x
266     x1
266     abc123
266     _abc123
266     intx
266     rangeVar
```

Why this is expected

All inputs match the identifier regex `LA*`. Strings like `intx` and `rangeVar` do not exactly match reserved words, so they remain identifiers.

3. `integers.cyl`

Goal

Confirm integer constants in decimal/octal/hex forms with common suffixes.

Observed output

```
267     0
267     10
267     123456
267     077
267     0x10
267     0XFF
267     10U
267     10LL
```

Why this is expected

All lexemes match one of the integer-constant patterns (hex via `0[xX]...`, non-zero decimal, or `0...` octal-style), optionally followed by `IS` suffixes.

4. `floats.cyl`

Goal Confirm floating constants, including scientific notation and hex floats.

Observed output

```
268     3.14
268     0.5
268     10.
268     1e10
268     1E-5
268     0x1.fp3
```

Why this is expected

Each lexeme matches one of the float rules (decimal with dot/exponent, or hex float using `P` exponent), so all are returned as `F_CONSTANT`.

5. `strings.cyl`

Goal Validate string literals, escapes, and adjacent string-literal concatenation.

Observed output

```
269     "hello"
"a" "b"
"line\nbreak"
```

Why this is expected

The string rule in `yapl.l` accepts one or more adjacent string literals separated by whitespace. Since whitespace includes newlines, multiple physical lines can be captured as a single `STRING_LITERAL` token; the extra line breaks are part of the printed lexeme.

6. `chars.cyl`

Goal Validate character literals with escapes.

Observed output

```
267     'a'
267     '\n'
267     '\t'
267     '\\'
```

Why this is expected

The character-literal rule accepts either a non-escape character or a valid escape sequence `ES` between single quotes, so all examples are tokenized as constants.

7. `comments.cyl`

Goal Ensure `//`, `#`, and `/*...*/` comments are skipped and do not produce tokens.

Observed output

```
308     int
266     x
59      ;
```

Why this is expected

Only the actual code tokens remain; comment text is fully consumed by the comment rules and never returned to the parser.

8. `bad_comments.cyl`

Goal Confirm that unterminated `/*...*/` comments are reported as fatal lexer errors.

Observed output

```
308     int
266     x
59      ;
yyerror: ***lexing terminated*** [lexer error]: ill-formed
    comment
exit_code=1
```

Why this is expected

If EOF is reached before closing `*/`, the `comment()` routine reports an ill-formed comment and terminates.

9. `extensions.cyl`

Goal Exercise **CYLite**-specific keywords in realistic statement shapes (`if/elif/else`, `for-in-range`, `foreach`, `try/except`, `pass`, `print`).

Observed output

```
323     if
40      (
266     x
41      )
259     pass
59      ;
323     if
40      (
266     x
41      )
259     pass
59      ;
258     elif
40      (
266     y
41      )
259     pass
59      ;
324     else
259     pass
59      ;
328     for
40      (
266     i
264     in
263     range
40      (
267     0
44      ,
267     10
41      )
41      )
262     print
40      (
266     i
41      )
59      ;
```

```
265     foreach
40      (
266     x
264     in
266     arr
41      )
123     {
262     print
40      (
266     x
41      )
59      ;
125     }
260     try
123     {
259     pass
59      ;
125     }
261     except
123     {
259     pass
59      ;
125     }
```

Why this is expected

All extension keywords are returned as dedicated tokens (not identifiers). Punctuation tokens appear with ASCII ids (e.g., `40` for `(`, `123` for `{`).

10. `invalid.cyl`

Goal Confirm that invalid characters are not silently ignored.

Observed output

```
308     int
266     x
61      =
267     5
yyerror: ***lexing terminated*** [lexer error]: invalid
    character '@'
exit_code=1
```

Why this is expected

After tokenizing the valid prefix, the catch-all rule `.` reports the invalid character and terminates immediately (non-zero exit code).

Sanity checks (what we verified). While running the above tests, we specifically sanity-checked a few high-signal cases:

- **keywords.cyl:** confirmed that `elif`, `try`, `foreach`, etc. are returned as keyword tokens (not as identifiers).
- **comments.cyl:** confirmed that `//...`, `#...`, and `/*...*/` are skipped and do not produce tokens.
- **invalid.cyl:** confirmed that an `@` triggers `***lexing terminated*** [lexer error]: invalid character '@'`.
- **bad_comments.cyl:** confirmed that an unclosed `/* ...` triggers `***lexing terminated*** [lexer error]: ill-formed comment`.

8 Current limitations (A2 scope)

- This phase focuses on correct tokenization and lexical error handling.
- The parser/semantic rules for constructs like `try/except` and `foreach` will be completed in later assignments.

9 Conclusion

In this assignment, we implemented the **CYLite** lexical analyzer using `lex` and ensured it matches our language specification for keywords, literals, operators/punctuation, and comment styles (including `#` single-line comments). We also made lexical errors explicit and fatal (invalid characters and unterminated block comments), so incorrect inputs do not silently pass through.

We validated the scanner using a token-dump driver (`dump_tokens.c`) and a focused set of test inputs under `tests/`, which helped us confirm both normal tokenization and error behavior. With the lexer stable, the next step is to complete and refine parsing support for the **CYLite**-specific constructs during A3.